LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5: RELASI ANTAR KELAS

Dibuat Oleh :

Nuevalen Refitra Alswando 1030724300008

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM SURABAYA

2026

1. TUJUAN PRAKTIKUM

1. Mengerti konsep dasar Object Oriented Programming
2. Memahami penggunaan kelas dalam pemrograman Java
3. Memahami hubungan antar kelas baik asosiasi, agregasi, dan komposisi
4. Dapat mengimplementasikan relasi antar kelas dalam program Java
5. Memahami diagram kelas dan notasi UML untuk relasi antar kelas

2. LANDASAN TEORI

2.1 Diagram Kelas

Diagram kelas merupakan sebuah diagram yang digunakan untuk memodelkan kelas-kelas yang digunakan di dalam sistem beserta hubungan antar kelas dalam sistem tersebut. Beberapa elemen penting dalam diagram kelas adalah kelas dan relasi antar kelas.

Simbol Access Modifier dalam Diagram Kelas:

Simbol	Keterangan
+	Public
-	Private
#	Protected

2.2 Jenis Hubungan Antar Kelas

2.2.1 Asosiasi

Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu sama lain, harus ada hubungan di antara kelas yang dapat diwakili oleh asosiasi.

Notasi UML: Garis lurus antara dua kelas

```
Class1 ———————— Class2
```

2.2.2 Agregasi

Agregasi merupakan hubungan antara dua kelas yang lebih ketat dan terdapat hubungan bagian (part) dan keseluruhan (whole). Kelas bagian dapat berdiri sendiri tanpa kelas keseluruhan.

Notasi UML: Garis dengan simbol wajik kosong (◇) di sisi whole

```
Class1 ◇──────────── Class2
(whole)            (part)
```

2.2.3 Komposisi

Komposisi merupakan bentuk khusus dari agregasi dimana terdapat hubungan antar kelas yang lebih kuat, karena yang menjadi part (bagian) tidak akan ada tanpa adanya kelas yang menjadi whole.

Notasi UML: Garis dengan simbol wajik penuh (◆) di sisi whole

```
Class1 ◆──────────── Class2
(whole)            (part)
```

2.2.4 Generalisasi (Inheritance)

Hubungan pewarisan dimana class merupakan turunan dari class lain.

Notasi UML: Garis dengan segitiga kosong mengarah ke superclass

```
Class1 ──────────▷ Class2
(subclass)       (superclass)
```

3. IMPLEMENTASI DAN PEMBAHASAN

3.1 Hubungan Asosiasi

Langkah Kerja:

1. Buka NetBeans
2. Buat Project baru: **Modul5_RelasiKelas**
3. Buat class **Mahasiswa**

File: Mahasiswa.java

```java
package modul05.Asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;  // Tambahan atribut nilai

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }

    public int getNilai() {
        return this.nilai;
    }
}
```

Penjelasan: Class Mahasiswa memiliki atribut private (nim, nama, nilai) dengan method getter dan setter untuk enkapsulasi.

File: Dosen.java

```java
package modul05.Asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    // Method asosiasi - memberikan nilai ke mahasiswa
    public void giveScore(Mahasiswa s, int nilai) {
        // Ini asosiasi, method milik class Mahasiswa dipanggil di class Dosen
        // tapi objek Student tidak menjadi atribut dari class Dosen
        s.setNilai(nilai);
    }

    // Method asosiasi - mengambil nilai dari mahasiswa
    public int getScore(Mahasiswa s) {
        // Ini asosiasi, method milik class Mahasiswa dipanggil di class Dosen
        return s.getNilai();
    }
}
```

Penjelasan: Class Dosen memiliki method giveScore() dan getScore() yang menerima objek Mahasiswa sebagai parameter. Ini menunjukkan hubungan asosiasi dimana Dosen berinteraksi dengan Mahasiswa tanpa memiliki objek Mahasiswa sebagai atribut permanen.

File: MainAsosiasi.java

```java
package modul05.Asosiasi;

public class MainAsosiasi {
    public static void main(String[] args) {
        // Membuat objek Mahasiswa
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");

        // Membuat objek Dosen
        Dosen d = new Dosen();
        d.setKodeDosen("D001");
        d.setNamaDosen("Dr. Ahmad");

        // Dosen memberikan nilai ke Mahasiswa (asosiasi)
        d.giveScore(m, nilai: 90);

        // Menampilkan informasi
        System.out.println("=== HUBUNGAN ASOSIASI ===");
        System.out.println("NIM: " + m.getNim());
        System.out.println("Nama: " + m.getNama());
        System.out.println("Dosen: " + d.getNamaDosen());
        System.out.println("Nilai: " + d.getScore(m));
    }
}
```

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=52962" -Dfile.encoding=UTF-8
=== HUBUNGAN ASOSIASI ===
NIM: 130118383
Nama: Budi
Dosen: Dr. Ahmad
Nilai: 90

Process finished with exit code 0
```

Penjelasan: Program utama mendemonstrasikan hubungan asosiasi dimana objek Dosen berinteraksi dengan objek Mahasiswa melalui method giveScore() dan getScore().

3.2 Hubungan Agregasi

File: Mahasiswa.java (versi agregasi)

main.java | Asosiasi\Mahasiswa.java | Dosen.java | MainAsosiasi.java | Agregasi\Mahasiswa.java | Jurusan.java | MainAgregasi.java

```java
package modul05.Asosiasi;

public class Mahasiswa {  2 usages  new *
    private String nim;  2 usages
    private String nama;  2 usages
    private int nilai;  // Tambahan atribut nilai  2 usages

    public void setNama(String nama) {  new *
        this.nama = nama;
    }

    public void setNim(String nim) {  new *
        this.nim = nim;
    }

    public void setNilai(int nilai) {  new *
        this.nilai = nilai;
    }

    public String getNama() {  new *
        return this.nama;
    }

    public String getNim() {  new *
        return this.nim;
    }

    public int getNilai() {  new *
        return this.nilai;
    }
}
```

Penjelasan: Class Mahasiswa dapat berdiri sendiri tanpa bergantung pada class Jurusan.

File: Jurusan.java

main.java | Asosiasi\Mahasiswa.java | Dosen.java | MainAsosiasi.java | Agregasi\Mahasiswa.java | Jurusan.java | MainAgregasi.java

m University\Semester 4\Pemrograman Berorientasi Objek\praktikum

```java
public class Dosen {  2 usages  new *
    private String kodeDosen;  2 usages
    private String namaDosen;  2 usages

    // Setter
    public void setKodeDosen(String kodeDosen) {  1 usage  new *
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {  1 usage  new *
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {  no usages  new *
        return this.kodeDosen;
    }

    public String getNamaDosen() {  1 usage  new *
        return this.namaDosen;
    }

    // Method asosiasi - memberikan nilai ke mahasiswa
    public void giveScore(Mahasiswa s, int nilai) {  1 usage  new *
        // Ini asosiasi, method milik class Mahasiswa dipanggil di class Dosen
        // tapi objek Student tidak menjadi atribut dari class Dosen
        s.setNilai(nilai);
    }

    // Method asosiasi - mengambil nilai dari mahasiswa
    public int getScore(Mahasiswa s) {  1 usage  new *
        // Ini asosiasi, method milik class Mahasiswa dipanggil di class Dosen
        return s.getNilai();
    }
}
```

Penjelasan: Class Jurusan memiliki atribut array dari class Mahasiswa. Ini menunjukkan hubungan agregasi dimana Mahasiswa merupakan bagian dari Jurusan, tetapi Mahasiswa dapat berdiri sendiri tanpa Jurusan.

File: MainAgregasi.java

```
main.java   Asosiasi\Mahasiswa.java   Dosen.java   MainAsosiasi.java ×   Agregasi\Mahasiswa.java   Jurusan.java   MainAgregasi.java

1   package modul05.Asosiasi;
2
3   public class MainAsosiasi {  new *
4       public static void main(String[] args) {  new *
5           // Membuat objek Mahasiswa
6           Mahasiswa m = new Mahasiswa();
7           m.setNim("130118383");
8           m.setNama("Budi");
9
10          // Membuat objek Dosen
11          Dosen d = new Dosen();
12          d.setKodeDosen("D001");
13          d.setNamaDosen("Dr. Ahmad");
14
15          // Dosen memberikan nilai ke Mahasiswa (asosiasi)
16          d.giveScore(m,  nilai: 90);
17
18          // Menampilkan informasi
19          System.out.println("=== HUBUNGAN ASOSIASI ===");
20          System.out.println("NIM: " + m.getNim());
21          System.out.println("Nama: " + m.getNama());
22          System.out.println("Dosen: " + d.getNamaDosen());
23          System.out.println("Nilai: " + d.getScore(m));
24      }
25  }
```

```
Run   MainAsosiasi ×

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=52078" -Dfile.encoding=UTF-8 -
=== HUBUNGAN ASOSIASI ===
NIM: 130118383
Nama: Budi
Dosen: Dr. Ahmad
Nilai: 90

Process finished with exit code 0
```

Penjelasan: Dalam agregasi, objek Mahasiswa dapat dibuat terlebih dahulu kemudian ditambahkan ke Jurusan. Objek Mahasiswa tetap dapat diakses meskipun tidak berada dalam Jurusan.

3.3 Hubungan Komposisi

File: CPU.java

```java
package modul05.Komposisi;

public class CPU {
    private String Merk;
    private int Kecepatan;

    public CPU(String m, int k) {
        this.Merk = m;
        this.Kecepatan = k;
    }

    public void DisplaySpecCPU() {
        System.out.println("CPU: " + this.Merk + ", " + this.Kecepatan + " GHz");
    }
}
```

Penjelasan: CPU adalah komponen yang menjadi bagian dari Komputer.

File: Monitor.java

```java
package modul05.Komposisi;

public class Monitor {
    private String Merk;

    public Monitor(String m) {
        this.Merk = m;
    }

    public void DisplaySpecMonitor() {
        System.out.println("Monitor: " + this.Merk);
    }
}
```

Penjelasan: Monitor adalah komponen yang menjadi bagian dari Komputer.

File: Mouse.java

```java
package modul05.Komposisi;

public class Mouse {
    private String Merk;
    private String Type;

    public Mouse(String m, String t) {
        this.Merk = m;
        this.Type = t;
    }

    public void DisplaySpecMouse() {
        System.out.println("Mouse: " + this.Merk + ", " + this.Type);
    }
}
```

Penjelasan: Mouse adalah komponen yang menjadi bagian dari Komputer.

File: Komputer.java

```java
package modul05.Komposisi;

public class Komputer {
    private CPU cpu;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String cpuMerk, int cpuSpeed, String monMerk, String mouseMerk, String mouseType) {
        // Komponen diciptakan saat instansiasi Komputer (komposisi)
        this.cpu = new CPU(cpuMerk, cpuSpeed);
        this.monitor = new Monitor(monMerk);
        this.mouse = new Mouse(mouseMerk, mouseType);
    }

    public void DisplaySpec() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        cpu.DisplaySpecCPU();
        monitor.DisplaySpecMonitor();
        mouse.DisplaySpecMouse();
    }
}
```

Penjelasan: Dalam komposisi, komponen (CPU, Monitor, Mouse) diciptakan saat instansiasi objek Komputer. Ketika objek Komputer musnah, komponen-komponennya juga ikut musnah. Ini adalah hubungan yang lebih kuat daripada agregasi.

File: MainKomposisi.java

```java
package modul05.Komposisi;

public class MainKomposisi {
    public static void main(String[] args) {
        // Membuat objek Komputer (komponen dibuat otomatis)
        Komputer k1 = new Komputer("Intel", 3, "Samsung", "Logitech", "Wireless");

        System.out.println("=== HUBUNGAN KOMPOSISI ===");
        k1.DisplaySpec();

        // Komponen tidak dapat diakses langsung dari luar
        // k1.cpu.DisplaySpecCPU(); // ERROR: cpu has private access

        System.out.println("\nKomponen hanya hidup bersama objek Komputer");
        System.out.println("Jika Komputer dihapus, komponen juga ikut dihapus");
    }
}
```

Penjelasan: Dalam komposisi, komponen hanya dapat diakses melalui objek Komputer. Komponen tidak memiliki lifecycle sendiri terpisah dari Komputer.

4. ANALISIS DAN PEMBAHASAN

4.1 Perbedaan Jenis Relasi

Jenis Relasi	Kekuatan	Lifecycle	Notasi UML	Contoh
Asosiasi	Lemah	Independen	————	Dosen - Mahasiswa
Agregasi	Sedang	Independen	◇————	Jurusan - Mahasiswa
Komposisi	Kuat	Dependen	◆————	Komputer - CPU
Generalisasi	Kuat	Dependen	————▷	Child - Parent

4.2 Kapan Menggunakan Setiap Relasi

Asosiasi:

- Ketika dua kelas perlu berkomunikasi
- Tidak ada hubungan kepemilikan
- Contoh: Dosen memberikan nilai ke Mahasiswa

Agregasi:

- Ketika ada hubungan bagian-keseluruhan
- Bagian dapat berdiri sendiri
- Lifecycle independen
- Contoh: Jurusan memiliki Mahasiswa

Komposisi:

- Ketika ada hubungan bagian-keseluruhan yang kuat
- Bagian tidak dapat berdiri sendiri
- Lifecycle dependen
- Contoh: Komputer memiliki CPU

Generalisasi:

- Ketika ada hubungan "is-a"
- Untuk inheritance/pewarisan
- Contoh: Mahasiswa adalah Person

4.3 Keuntungan Memahami Relasi Antar Kelas

1. **Desain yang lebih baik**: Memahami hubungan antar kelas membantu membuat desain sistem yang lebih terstruktur
2. **Maintainability**: Kode lebih mudah dipelihara ketika relasi didefinisikan dengan jelas
3. **Reusability**: Kelas dapat digunakan kembali dalam konteks yang berbeda

4. **Scalability**: Sistem lebih mudah dikembangkan dengan memahami relasi yang ada

5. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. **Diagram Kelas** adalah alat untuk memodelkan kelas-kelas dalam sistem beserta hubungan antar kelas menggunakan notasi UML.
2. **Asosiasi** adalah hubungan paling dasar antara dua kelas dimana objek dari satu kelas berinteraksi dengan objek dari kelas lain tanpa kepemilikan.
3. **Agregasi** adalah hubungan "has-a" yang lemah dimana objek bagian dapat berdiri sendiri tanpa objek keseluruhan. Lifecycle objek bagian independen.
4. **Komposisi** adalah hubungan "has-a" yang kuat dimana objek bagian tidak dapat berdiri sendiri tanpa objek keseluruhan. Lifecycle objek bagian dependen pada objek keseluruhan.
5. **Generalisasi** adalah hubungan inheritance/pewarisan dimana subclass mewarisi atribut dan method dari superclass (hubungan "is-a").
6. **Notasi UML** membantu dalam mendokumentasikan dan mengkomunikasikan desain sistem kepada developer lain.
7. **Pemilihan relasi yang tepat** sangat penting untuk menciptakan desain sistem yang baik, maintainable, dan scalable.
8. **Enkapsulasi** tetap penting dalam semua jenis relasi untuk melindungi data internal kelas.
9. **Implementasi Java** dari setiap relasi berbeda:
    - Asosiasi: parameter method
    - Agregasi: atribut dengan objek yang dibuat terpisah
    - Komposisi: atribut dengan objek yang dibuat dalam constructor
10. **Pemahaman relasi antar kelas** adalah fondasi penting dalam Object Oriented Design untuk menciptakan sistem yang terstruktur dan mudah dikembangkan.